Formalizing the Lie–Trotter Product Formula and Commutator-Scaling Error Bounds in Lean 4

April 15, 2026

Abstract

We present a machine-checked proof of the Lie–Trotter product formula $e^{A+B} = \lim_{n\to\infty}(e^{A/n}\, e^{B/n})^n$ for elements A, B in a complete normed algebra, formalized in Lean 4 with the Mathlib library. Beyond the basic convergence theorem with an explicit $O(1/n)$ error bound, our development includes the symmetric Strang splitting with $O(1/n^2)$ convergence, multi-operator generalizations, a fourth-order Suzuki integrator, and—most significantly—commutator-scaling error bounds matching those of Childs et al. (2021), where the product $\|A\|\,\|B\|$ is replaced by the commutator norm $\|[B,\, A]\|$. The commutator-scaling proofs introduce a Duhamel/FTC-based technique that avoids ODE uniqueness theory entirely. The formalization comprises over 4 300 lines of Lean 4 with zero `sorry` axioms, building against current Mathlib and filling several gaps in the library's treatment of the matrix exponential.

1 Introduction

Product formulas are among the most widely used tools in quantum simulation. The Lie–Trotter formula

$$e^{A+B} = \lim_{n\to\infty}\left(e^{A/n}\, e^{B/n}\right)^n \tag{1}$$

decomposes the exponential of a sum into a product of simpler exponentials, enabling the simulation of composite quantum systems by sequentially simulating each term of the Hamiltonian [3, 10]. Higher-order variants—the symmetric Strang splitting [6] and Suzuki's recursive hierarchy [7]—achieve faster convergence by exploiting symmetry and commutator cancellation. Rigorous error bounds for these formulas are critical for resource estimation in quantum algorithms [1].

The quality of these error bounds directly affects the efficiency of quantum algorithms. The standard bound scales with the product $\|A\|\,\|B\|$, which treats A and B as unrelated. Childs et al. [1] initiated a program of *commutator-scaling* bounds, showing that the leading error is controlled by the commutator norm $\|[B,\, A]\|$ rather than the product $\|A\|\,\|B\|$. For spatially local Hamiltonians $H = \sum_j h_j$ typical of many-body physics, most pairs of terms commute, so $\sum_{j<k}\|[h_j,\, h_k]\|$ can be polynomially smaller than $\sum_{j<k}\|h_j\|\,\|h_k\|$. These tighter bounds translate directly into fewer Trotter steps—and hence fewer quantum gates—for a given simulation accuracy.

In this work, we present a complete formalization of the Lie–Trotter product formula and its higher-order extensions in Lean 4, an interactive theorem prover with a small trusted kernel [2]. The Mathlib library provides a large corpus of formalized mathematics [8], including the normed algebra exponential, upon which our development builds.

Proofs of product formula error bounds involve long chains of norm estimates in non-commutative algebras, where each step interacts with the previous through submultiplicativity, the triangle inequality, and exponential series manipulation. A single bookkeeping error—a missing factor, a

flipped sign in a commutator—can silently propagate and invalidate the final bound. Formal verification forces every intermediate step to be explicit and machine-checked, making it a natural fit for this domain.

Our formalization proves the following hierarchy of results:

1. **First-order Lie–Trotter** (Section 3.5): $\left\|(e^{A/n}\, e^{B/n})^n - e^{A+B}\right\| \le C/n$ with explicit constant $C = 2\,\|A\|\,\|B\|\,e^{\|A\|+\|B\|} + 1$.

2. **Strang splitting** (Section 3.7): $\left\|(e^{A/2n}\, e^{B/n}\, e^{A/2n})^n - e^{A+B}\right\| \le C'/n^2$.

3. **Multi-operator extensions**: $(e^{A_1/n} \cdots e^{A_m/n})^n \to e^{A_1+\cdots+A_m}$ and the palindromic (Strang) variant, both with explicit error bounds.

4. **Fourth-order Suzuki integrator**: $S_4(t) = S_2(pt)^2\, S_2((1-4p)t)\, S_2(pt)^2$ with $O(1/n^2)$ convergence.

5. **Commutator-scaling bounds** (Section 3.6):

$$\left\|e^{tB}\, e^{tA} - e^{t(A+B)}\right\| \le \tfrac{1}{2}\,\|[B,\,A]\|\; t^2\, e^{t(\|A\|+3\|B\|)}, \tag{2}$$

$$\left\|S_2(t) - e^{tH}\right\| \le \left(\tfrac{1}{12}\,\|[B,\,[B,\,A]]\| + \tfrac{1}{24}\,\|[A,\,[A,\,B]]\|\right) t^3, \tag{3}$$

   matching the commutator-scaling program of Childs et al. [1].

The entire development comprises 13 Lean source files totaling over 4 300 lines, with zero `sorry` axioms. Our Lean 4 formalization is available at `github.com/Jue-Xu/Lean-Trotter`. Our main contributions are:

- The *first formalization* of the Lie–Trotter product formula in any proof assistant, including its higher-order and multi-operator extensions.

- A *Duhamel/FTC-based proof technique* for commutator-scaling bounds that avoids ODE uniqueness theory (Gronwall's inequality, Picard iteration), requiring only the fundamental theorem of calculus for Bochner integrals.

- Machine-checked *commutator-scaling error bounds* at first and second order, with double-commutator coefficients matching the theoretical literature.

- Identification and filling of several *gaps in* Mathlib, notably the inequality $\|e^a\| \le e^{\|a\|}$ for general Banach algebras.

To our knowledge, no prior formalization of the Lie–Trotter product formula exists in any proof assistant. Mathlib contains the identity $e^{a+b} = e^a\, e^b$ for commuting elements (`exp_add_of_commute`), but the non-commuting convergence theorem—which is a limit statement, not an algebraic identity—is new.

The remainder of this paper is organized as follows. Section 2 reviews the mathematical background on product formulas and the Lean/Mathlib ecosystem. Section 3 presents the proof architecture, including the modular decomposition and the Duhamel technique for commutator scaling. Section 4 discusses the formalization in Lean 4, with representative code excerpts and design decisions. Section 5 reflects on the development process, including AI-assisted formalization and lessons learned. Section 6 concludes with future directions.

2 Mathematical Background

2.1 The Lie–Trotter product formula

Let $\mathfrak{A}$ be a unital Banach algebra over $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$ with norm $\|\cdot\|$ satisfying $\|1\| = 1$ and $\|ab\| \leq \|a\| \, \|b\|$. The exponential

$$e^a = \sum_{k=0}^{\infty} \frac{a^k}{k!}$$

converges absolutely for every $a \in \mathfrak{A}$.

Trotter [10] proved that for bounded operators A, B,

$$e^{A+B} = \lim_{n\to\infty} \left(e^{A/n} \, e^{B/n}\right)^n.$$

The standard proof proceeds in two steps:

1. *Telescoping*: the identity $X^n - Y^n = \sum_{k=0}^{n-1} X^k (X-Y) Y^{n-1-k}$ gives $\|X^n - Y^n\| \leq n \, \|X - Y\| \, (\max\{\|X\|, \|Y$
2. *Step error*: the single-step approximation satisfies $\left\|e^a \, e^b - e^{a+b}\right\| = O(\|a\| \, \|b\|)$.

Setting $X = e^{A/n} \, e^{B/n}$ and $Y = e^{(A+B)/n}$, the $O(1/n^2)$ step error is amplified by the factor n from telescoping, yielding $O(1/n)$ overall convergence.

2.2 Higher-order splitting methods

The symmetric *Strang splitting* [6] uses

$$S_2(t) = e^{tA/2} \, e^{tB} \, e^{tA/2}$$

to achieve $O(t^3)$ local error and $O(1/n^2)$ global convergence. Suzuki [7] showed that higher-order integrators can be constructed recursively: the fourth-order formula

$$S_4(t) = S_2(pt)^2 \, S_2\big((1-4p)t\big) \, S_2(pt)^2 \qquad \text{with } p = \frac{1}{4 - 4^{1/3}}$$

achieves $O(t^5)$ local error.

Both formulas generalize to multiple operators $A_1, \ldots, A_m$. The first-order product $(e^{A_1/n} \cdots e^{A_m/n})^n$ converges at rate $O(1/n)$, while the palindromic product

$$S_2^{(m)}(t) = e^{tA_1/2} \, e^{tA_2/2} \cdots e^{tA_m} \cdots e^{tA_2/2} \, e^{tA_1/2}$$

achieves $O(1/n^2)$.

2.3 Commutator-scaling error bounds

The standard step error $\left\|e^a \, e^b - e^{a+b}\right\| \leq 2 \, \|a\| \, \|b\| \, e^{\|a\|+\|b\|}$ treats A and B as unrelated. When A and B nearly commute—as is typical for spatially local Hamiltonians—the error is much smaller.

Childs et al. [1] showed that the leading error term is controlled by the commutator $[B, A] = BA - AB$ rather than the product $\|A\| \, \|B\|$. At first order,

$$\left\|e^{tB} \, e^{tA} - e^{t(A+B)}\right\| \leq \frac{\|[B, A]\|}{2} \, t^2 \, e^{t(\|A\|+3\|B\|)}. \tag{4}$$

For the Strang splitting, the leading commutator $[B, A]$ cancels by symmetry, and the error is controlled by double commutators:

$$\left\|S_2(t) - e^{tH}\right\| \leq \left(\frac{\|[B, [B, A]]\|}{12} + \frac{\|[A, [A, B]]\|}{24}\right) t^3. \tag{5}$$

These commutator-scaling bounds are physically significant: for a lattice Hamiltonian $H = \sum_j h_j$ where each h_j acts on a few neighboring sites, $\|[h_j, h_k]\| = 0$ whenever the supports of h_j and h_k are disjoint, so the commutator-based bound can be polynomially tighter than the naive product bound [1].

2.4 Lean 4 and Mathlib

Lean 4 is an interactive theorem prover and programming language based on dependent type theory [2]. Proofs and programs share the same language, and correctness is guaranteed by a small trusted kernel that independently type-checks every proof term.

The Mathlib library [8] provides formalized mathematics including abstract algebra, analysis, and topology. For our development, the key components are:

- `NormedRing`, `NormedAlgebra`: typeclasses for normed rings and algebras with submultiplicative norms.
- `NormedSpace.exp`: the exponential function $a \mapsto \sum_k a^k/k!$ in a complete normed algebra.
- `intervalIntegral`: the Bochner integral over $[a, b]$, with the fundamental theorem of calculus.
- `HasDerivAt`: the Fréchet derivative, with product rules and chain rules for normed algebra-valued functions.

Related formalization work. Formal verification has recently gained traction in mathematical physics and quantum information. Zhao and Yu [11] formalized robust CHSH rigidity in Lean 4, discovering a gap in the original proof of McKague (2012) during the formalization process. Meiburg et al. [4] formalized a generalized quantum Stein's lemma, and Tooby-Smith [9] formalized the stability of the two-Higgs-doublet potential, also identifying a literature error. These projects share the pattern of using formal verification not merely to certify known results but to expose subtle gaps in published proofs. Our work extends this program to quantum simulation, providing the first machine-checked proof of any product formula for operator exponentials.

3 Proof Architecture

Our formalization is organized into five independent tracks that compose to yield the main theorem and its extensions. Figure 1 shows the dependency structure. We present the mathematical arguments here; the Lean formalization is discussed in Section 4.

3.1 Modular decomposition

The proof of the Lie–Trotter formula decomposes into four independent components:

Track 1 (Algebraic): The telescoping identity and its norm bound.

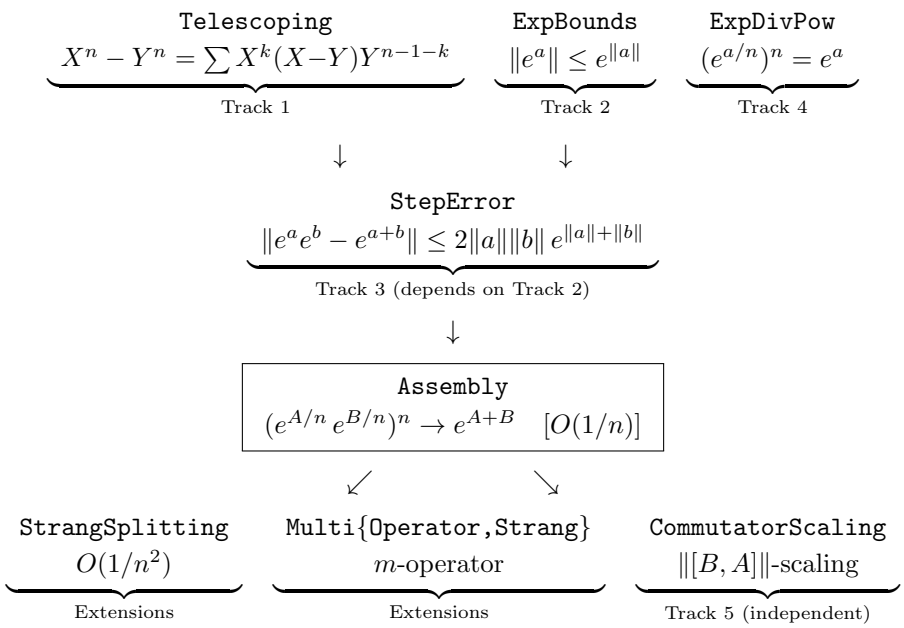

Figure 1: Dependency structure of the formalization. Tracks 1–4 compose into the main theorem; extensions branch from the assembly or develop independently.

Track 2 (Analytic): Exponential series remainder estimates.

Track 3 (Step error): The single-step approximation bound $\left\|e^a\, e^b - e^{a+b}\right\| \le 2\,\|a\|\,\|b\|\; e^{\|a\|+\|b\|}$.

Track 4 (Connecting): The identity $(e^{a/n})^n = e^a$.

Tracks 1–4 are combined in an assembly step to yield the convergence rate, from which the limit theorem follows by a standard ε-N argument. The commutator-scaling bounds (Track 5) are an independent extension that refines the step error.

3.2 Telescoping and norm bounds

The algebraic identity

$$X^n - Y^n = \sum_{k=0}^{n-1} X^k\,(X-Y)\,Y^{n-1-k} \tag{6}$$

is proved by induction on n. The key step is factoring Y out of the inner sum to match the inductive hypothesis: writing $Y^{m-k} = Y^{m-1-k} \cdot Y$ converts $\sum_{k<m} X^k(X-Y)Y^{m-k}$ into $(\sum_{k<m} X^k(X-Y)Y^{m-1-k}) \cdot Y$.

The norm bound follows by the triangle inequality applied to each summand:

$$\|X^n - Y^n\| \le n\;\|X-Y\|\;M^{n-1}, \qquad M = \max\{\|X\|\,,\|Y\|\}. \tag{7}$$

3.3 Exponential series estimates

We establish four bounds on the exponential in a complete normed algebra:

$$\|e^a\| \le e^{\|a\|}, \tag{8}$$

$$\|e^a - 1\| \le e^{\|a\|} - 1, \tag{9}$$

$$e^x - 1 - x \le \tfrac{x^2}{2}\, e^x \quad (x \ge 0), \tag{10}$$

$$\|e^a - 1 - a\| \le \tfrac{\|a\|^2}{2}\, e^{\|a\|}. \tag{11}$$

The proofs use the power series representation $e^a = \sum_{n=0}^{\infty} a^n/n!$. Bounds (8)–(9) follow from $\|\sum a^n/n!\| \le \sum \|a\|^n/n! = e^{\|a\|}$ with appropriate index shifts. Bound (10) uses the termwise estimate $x^{n+2}/(n+2)! \le (x^2/2) \cdot x^n/n!$ via the factorial inequality $2 \cdot n! \le (n+2)!$. Bound (11) combines (10) with the same tsum comparison technique as (8).

Remark. Bound (8) was not available in Mathlib for general Banach algebras; Mathlib only had the specialized result `Complex.norm_exp_le_exp_norm` for $\mathbb{C}$. Our proof from the power series fills this gap.

3.4 Step error via algebraic factorization

The single-step bound is the most delicate component. Rather than expanding $e^a\, e^b$ and e^{a+b} to second order and bounding cross-terms directly (which is tractable on paper but unwieldy in a proof assistant due to non-commutative bookkeeping), we use an algebraic factorization:

$$e^a\, e^b - e^{a+b} = (e^a - 1)(e^b - 1) - \big(e^{a+b} - e^a - e^b + 1\big). \tag{12}$$

The first term is bounded by $(e^{\|a\|} - 1)(e^{\|b\|} - 1)$ via (9). For the second (cross) term, we prove by induction on the power m that

$$\|(a+b)^m - a^m - b^m\| \le (\|a\| + \|b\|)^m - \|a\|^m - \|b\|^m \qquad (m \ge 1), \tag{13}$$

using the non-commutative recurrence $(a+b)^{m+1} - a^{m+1} - b^{m+1} = (a+b)\big((a+b)^m - a^m - b^m\big) + ab^m + ba^m$. Summing over the exponential series, the cross term is also bounded by $(e^{\|a\|} - 1)(e^{\|b\|} - 1)$, so by the triangle inequality:

$$\left\|e^a\, e^b - e^{a+b}\right\| \le 2(e^{\|a\|} - 1)(e^{\|b\|} - 1) \le 2\,\|a\|\,\|b\|\; e^{\|a\| + \|b\|}, \tag{14}$$

where the last step uses $(e^s - 1)(e^t - 1) \le st\, e^{s+t}$ for $s, t \ge 0$.

Remark. The factorization (12) was discovered during formalization: the standard second-order expansion approach required tracking non-commutative multinomial cross-terms, which was intractable in Lean. The algebraic factorization reduces the problem to two independent bounds, each handled by series comparison.

3.5 Assembly and convergence

Setting $P = e^{A/n}\, e^{B/n}$ and $Q = e^{(A+B)/n}$:

1. $Q^n = e^{A+B}$ by the identity $(e^{a/n})^n = e^a$ (`exp_div_pow`).
2. $\|P - Q\| \le 2\,\|A\|\,\|B\|\,/n^2 \cdot e^{(\|A\| + \|B\|)/n}$ by the step error with $a = A/n$, $b = B/n$.

3. $\max\{\|P\|, \|Q\|\} \le e^{(\|A\|+\|B\|)/n}$ by (8).
4. Telescoping gives $\|P^n - Q^n\| \le n \cdot \|P - Q\| \cdot M^{n-1}$.
5. The factors $e^{(\|A\|+\|B\|)/n}$ combine: $e^{s/n} \cdot (e^{s/n})^{n-1} = (e^{s/n})^n = e^s$, yielding

$$\left\|(e^{A/n}\, e^{B/n})^n - e^{A+B}\right\| \le \frac{2\,\|A\|\,\|B\|\; e^{\|A\|+\|B\|}}{n}.$$

The main theorem $\lim_{n\to\infty}(e^{A/n}\, e^{B/n})^n = e^{A+B}$ follows by a standard ε-N argument using the Archimedean property.

3.6 Duhamel approach to commutator scaling

The commutator-scaling bound (2) replaces the product $\|A\|\,\|B\|$ with $\|[B,\, A]\|$. Our proof uses a variation-of-parameters (Duhamel) technique based on the fundamental theorem of calculus, avoiding ODE uniqueness theory entirely.

Integral error representation. Define the "conjugated Trotter product"

$$w(\tau) = e^{-\tau(A+B)}\, e^{\tau B}\, e^{\tau A}. \tag{15}$$

By the product rule for Fréchet derivatives in a normed algebra,

$$w'(\tau) = e^{-\tau(A+B)}\left(e^{\tau B}\, A - A\, e^{\tau B}\right) e^{\tau A} = e^{-\tau(A+B)}\,[e^{\tau B},\, A]\, e^{\tau A}. \tag{16}$$

The key cancellation is that the "free" terms involving $A+B$, B, and A sum to zero: $-(A+B) + B + A = 0$. The FTC-2 gives

$$e^{tB}\, e^{tA} - e^{t(A+B)} = \int_0^t e^{(t-\tau)(A+B)}\,[e^{\tau B},\, A]\, e^{\tau A}\, d\tau, \tag{17}$$

after left-multiplying by $e^{t(A+B)}$ (which commutes with itself).

Extracting the commutator. The integrand contains $[e^{\tau B},\, A] = e^{\tau B}A - Ae^{\tau B}$, which involves the *exponential* of B. To extract the bare commutator $[B,\, A]$, we apply FTC-2 a second time to the conjugation map $s \mapsto e^{sB}\, A\, e^{-sB}$:

$$e^{\tau B}\, A\, e^{-\tau B} - A = \int_0^\tau e^{sB}\,[B,\, A]\, e^{-sB}\, ds. \tag{18}$$

This gives $\left\|e^{\tau B}\, A\, e^{-\tau B} - A\right\| \le \|[B,\, A]\|\; |\tau|\; e^{2|\tau|\|B\|}$ by bounding the integrand and using $\left\|\int_0^\tau f\right\| \le C\,|\tau|$. Right-multiplying by $e^{\tau B}$ yields $\left\|[e^{\tau B},\, A]\right\| \le \|[B,\, A]\|\; |\tau|\; e^{3|\tau|\|B\|}$.

Final bound. Substituting into (17) and bounding the integral:

$$\left\|e^{tB}\, e^{tA} - e^{t(A+B)}\right\| \le \int_0^t e^{(t-\tau)(\|A\|+\|B\|)} \cdot \|[B,\, A]\|\; \tau\, e^{3\tau\|B\|} \cdot e^{\tau\|A\|}\, d\tau.$$

Simplifying the exponential factors and evaluating $\int_0^t \tau\, d\tau = t^2/2$ gives the claimed bound (2).

Remark. The Duhamel approach requires only the product rule for Fréchet derivatives, the FTC-2 for Bochner integrals, and basic norm estimates—all available in Mathlib. The alternative approach via Gronwall's inequality would require ODE existence/uniqueness theory, adding approximately 40 lines of additional Lean infrastructure.

3.7 Second-order Strang commutator scaling

For the Strang splitting $S_2(t) = e^{tA/2}\, e^{tB}\, e^{tA/2}$, we prove

$$\left\|S_2(t) - e^{tH}\right\| \le \frac{\|[B,\,[B,\,A]]\|}{12}\, t^3 + \frac{\|[A,\,[A,\,B]]\|}{24}\, t^3, \tag{19}$$

where $H = A + B$ and we additionally require that A and B are *skew-adjoint* ($a^* = -a$), so that $\left\|e^{ta}\right\| = 1$.

The proof applies the same Duhamel strategy to $w(\tau) = e^{-\tau H}\, S_2(\tau)$, but now requires:

1. A *4-factor product rule* for $S_2(\tau) = e^{\tau A/2}\, e^{\tau B}\, e^{\tau A/2}$, yielding $w'(\tau) = e^{-\tau H}\, \mathcal{T}_2(\tau)\, S_2(\tau)$, where $\mathcal{T}_2$ involves the commutators $[A/2,\, e^{\tau B}]$ and $[e^{\tau B},\, A/2]$.
2. *First-order cancellation*: the leading commutator terms cancel exactly: $[A/2,\, B] + [B,\, A/2] = 0$.
3. A *double FTC* on $s \mapsto e^{sB}\, A\, e^{-sB}$, expanding to second order to extract $[B,\,[B,\,A]]$.
4. The *"subtract-constant-at-τ" trick*: to bound $\int_0^\tau F(s)\, ds - \tau\, F(\tau)$, rewrite as $\int_0^\tau (F(s) - F(\tau))\, ds$ and bound $\|F(s) - F(\tau)\| \le C_A(\tau - s)$. This avoids Fubini's theorem or integration by parts.

The 4-factor product rule. Write $A' = A/2$ and $H = A + B$. The conjugated Strang product

$$w(\tau) = e^{-\tau H}\, e^{\tau A'}\, e^{\tau B}\, e^{\tau A'}$$

has four exponential factors, each contributing a derivative term via the Leibniz rule. Using $d/d\tau[e^{\tau X}] = X\, e^{\tau X}$ and collecting:

$$w'(\tau) = e^{-\tau H}\, \mathcal{T}_2(\tau)\, S_2(\tau), \tag{20}$$

where the *Strang residual* is

$$\mathcal{T}_2(\tau) = \big(e^{\tau A'}\, B\, e^{-\tau A'} - B\big) + e^{\tau A'}\big(e^{\tau B}\, A'\, e^{-\tau B} - A'\big)e^{-\tau A'}.$$

The key algebraic step is verifying that the "free" derivative terms $-H + A' + B + A'$ sum to zero (since $A' + A' = A$ and $H = A + B$). In Lean, the sum of the four derivative contributions is factored as $-(e^{-\tau H}) \cdot (H - A' - A' - B) \cdot S_2(\tau)$, where $H - A' - A' - B = 0$ is discharged by `abel`. The free-ring factoring is handled by `noncomm_ring`.

Coefficient conversion. Since $A' = A/2$, the double commutators in the final bound involve half-scaled operators: $[B,\,[B,\,A']] = \frac{1}{2}[B,\,[B,\,A]]$ and $[A',\,[A',\,B]] = \frac{1}{4}[A,\,[A,\,B]]$. The scalar factors are extracted through $[cX,\,Y] = c\,[X,\,Y]$ (valid for central scalars) using `Algebra.smul_mul_assoc` and `Algebra.mul_smul_comm`, then `norm_smul`. Combined with the integration constants (1/6 from the cubic integral and 1/2 from the double FTC), this produces the final coefficients 1/12 and 1/24 in (19).

4 Formalization in Lean 4

4.1 Project structure and statistics

The formalization is organized into 13 Lean source files, listed in Table 1. Each file is self-contained with explicit imports, enabling parallel development and independent verification.

The development builds against Lean 4.29.0 and Mathlib (commit `06a46dae`). Build time is under 3 seconds on a standard laptop after Mathlib oleans are cached (`lake exe cache get`).

File	Lines	Content
`Telescoping.lean`	106	Algebraic telescoping + norm bound
`ExpBounds.lean`	212	Exponential series remainder estimates
`StepError.lean`	674	Quadratic step error (algebraic factorization)
`ExpDivPow.lean`	54	$(e^{a/n})^n = e^a$
`Assembly.lean`	146	$O(1/n)$ convergence + main theorem
`StrangSplitting.lean`	492	Strang splitting, $O(1/n^2)$
`MultiOperator.lean`	288	Multi-operator first-order
`MultiStrang.lean`	428	Multi-operator Strang
`Suzuki4.lean`	466	Fourth-order Suzuki integrator
`CommutatorScaling.lean`	379	Duhamel commutator-scaling
`MultiCommutatorScaling.lean`	128	Multi-operator comm. scaling
`StrangCommutatorScaling.lean`	733	Strang double-commutator scaling
`MultiStrangCommutatorScaling.lean`	174	Multi-op. Strang comm. scaling
Total	**4 280**	**0 `sorry` axioms**

Table 1: Source files in the formalization.

4.2 Type-level setup

Our theorems are stated for a general complete normed algebra $\mathfrak{A}$ over a field $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$:

```
variable {K : Type*} [RCLike K]
variable {A : Type*} [NormedRing A] [NormedAlgebra K A]
  [NormOneClass A] [CompleteSpace A]
```

The typeclass `NormOneClass` requires $\|1\| = 1$, which is needed for `norm_pow_le` in current Mathlib. The main theorem is (`lie_trotter`):

```
theorem lie_trotter (A B : A) :
    Filter.Tendsto
      (fun n : N => (exp ((n : K)^{-1} * A) *
                     exp ((n : K)^{-1} * B)) ^ n)
      atTop (nhds (exp (A + B)))
```

The `include` pattern. Since `NormedSpace.exp` no longer takes a field parameter in recent Mathlib, the variable $\mathbb{K}$ does not appear in the type of theorems involving `exp`. Lean 4 drops unused type-level variables, so we must write `include K in` before each lemma whose proof (but not statement) requires $\mathbb{K}$ (e.g., `exp_tsum_form`):

```
include K in
lemma exp_tsum_form (a : A) :
    exp a = \sum' n, (((Nat.factorial n : K)^{-1}) * a ^ n : A) := by ...
```

4.3 Telescoping in Lean

The telescoping identity (6) is proved by induction on n using `Finset.sum_range_succ` to peel off the last term (`telescoping_direct`):

```
theorem telescoping_direct (X Y : R) (n : N) :
    (sum k in Finset.range n,
      X ^ k * (X - Y) * Y ^ (n - 1 - k)) =
      X ^ n - Y ^ n := by
  induction n with
  | zero => simp
  | succ m ih =>
    rw [Finset.sum_range_succ]
    -- Factor Y from the inner sum to get the
    -- inductive hypothesis
    ...
    noncomm_ring [pow_succ]
```

The final step uses `noncomm_ring`, which handles polynomial identities in non-commutative rings. The standard `ring` tactic silently fails on non-commutative goals—it produces an unsolved subgoal rather than an error, which cost us significant debugging time early in the project.

4.4 Exponential bounds: filling Mathlib gaps

The bound $\|e^a\| \leq e^{\|a\|}$ was not available in Mathlib for general Banach algebras. Our proof works from the tsum representation (`norm_exp_le`):

```
theorem norm_exp_le (a : A) :
    ||exp a|| <= Real.exp ||a|| := by
  rw [exp_tsum_form (K := K) a]
  calc ||sum' n, ...||
      <= sum' n, ||...||      -- norm_tsum_le_tsum_norm
    _ <= sum' n, ||a||^n / n! -- norm_exp_term_le
    _ = Real.exp ||a||         -- Real.hasSum_exp
```

The proof chains three standard ingredients: pulling the norm inside the tsum, bounding each term $\|a^n/n!\| \leq \|a\|^n/n!$ via submultiplicativity, and recognizing the resulting series as $e^{\|a\|}$.

4.5 Step error: algebraic factorization in Lean

The factorization (12) is verified by `noncomm_ring`. The cross-term bound (13) is proved by induction using the non-commutative recurrence (`norm_pow_add_sub_pow_sub_pow`):

```
theorem norm_pow_add_sub_pow_sub_pow (a b : A) :
    forall m : N, 1 <= m ->
    ||(a + b) ^ m - a ^ m - b ^ m|| <=
      (||a|| + ||b||) ^ m - ||a|| ^ m - ||b|| ^ m := by
  intro m hm
  induction m with
  | zero => omega
  | succ n ih => ...
```

At 674 lines, `StepError.lean` is the longest file in the basic Lie–Trotter development (excluding commutator-scaling extensions). The inductive cross-term bound accounts for roughly half of this length, because each step requires explicit `norm_mul_le` applications and `gcongr` for the inductive comparison.

4.6 Commutator scaling: calculus in Lean

The commutator-scaling proofs are the most technically demanding part of the formalization, as they require doing analysis—derivatives, interval integrals, FTC-2—in a non-commutative Banach algebra.

Derivative computations. The derivative $d/d\tau[e^{\tau B}] = B\, e^{\tau B}$ is provided by `hasDerivAt_exp_smul_const'` in Mathlib. The product rule `HasDerivAt.mul` extends to normed algebras. One subtlety: $(-\tau) \bullet B$ and $\tau \bullet (-B)$ are definitionally distinct but equal; we must normalize before applying derivative lemmas (`hasDerivAt_exp_conj`):

```
lemma hasDerivAt_exp_conj (A B : A) (s : R) :
    HasDerivAt
      (fun u => exp (u * B) * A * exp ((-u) * B))
      (exp (s * B) * (B * A - A * B) *
       exp ((-s) * B)) s := by
  simp_rw [show forall u : R, (-u) * B = u * (-B)
    from fun u => by rw [neg_smul, smul_neg]]
  ...
```

FTC-2 for integral representations. The conjugation identity (18) and the integral error representation (17) are proved by `integral_eq_sub_of_hasDerivAt` (`exp_conj_sub_eq_integral`):

```
theorem exp_conj_sub_eq_integral (A B : A) (t : R) :
    exp (t * B) * A * exp ((-t) * B) - A =
      integral s in (0 : R)..t,
        exp (s * B) * (B * A - A * B) *
        exp ((-s) * B) := by
  have hderiv : forall u in Set.uIcc 0 t,
      HasDerivAt ... := fun u _ =>
    hasDerivAt_exp_conj A B u
  have hint : IntervalIntegrable ... :=
    (continuous_exp_conj_deriv A B).intervalIntegrable 0 t
  exact (integral_eq_sub_of_hasDerivAt hderiv hint).symm
```

Pulling constants through integrals. In a `NormedRing`, left-multiplication by a fixed element c is *not* a scalar multiplication—it is ring multiplication. To derive $c \cdot \int f = \int c \cdot f$, we use `ContinuousLinearMap.intervalIntegral_comp_comm` with $L =$ `ContinuousLinearMap.mul` $\mathbb{R}\ \mathfrak{A}\ c$, which views left-multiplication as a continuous linear map.

Working over $\mathbb{R}$. The commutator-scaling files work over `NormedAlgebra` $\mathbb{R}\ \mathfrak{A}$ directly, rather than a general $\mathbb{K}$. The reason is that the `HasDerivAt/intervalIntegral` machinery requires `SMul` $\mathbb{R}\ \mathfrak{A}$, which is *not* automatically synthesized from `[RCLike K] [NormedAlgebra K A]`. For applications with $\mathbb{K} = \mathbb{C}$, one can use `NormedAlgebra.restrictScalars` $\mathbb{R}\ \mathbb{C}\ \mathfrak{A}$.

4.7 Multi-operator and Suzuki extensions

The multi-operator formulas are proved by induction on the list of operators. The palindromic product is defined recursively (`palindromicProd`):

```
def palindromicProd (ops : List A) (t : R) : A :=
  match ops with
  | [] => 1
  | [a] => exp (t * a)
  | a :: rest =>
    exp ((t / 2) * a) *
    palindromicProd rest t *
    exp ((t / 2) * a)
```

The inductive step peels off the outermost $e^{tA_1/2}$ factors and applies the two-operator Strang bound to the pair $(A_1, A_2 + \cdots + A_m)$, then recurses on the inner product.

5 Discussion

5.1 AI assistance in the development process

We used LLM-based coding assistants (Claude Code, Anthropic) to accelerate formalization, in the same general direction as recent work on agentic autoformalization [5].

Our workflow followed a *sorry-driven development* pattern:

1. State all theorem signatures with `sorry` placeholders.
2. Verify that the signatures compose correctly (the main theorem type-checks with all intermediates as `sorry`).
3. Fill `sorry`s bottom-up, from leaves of the dependency DAG to the root.

This approach worked well with LLM agents: independent leaves (e.g., the four exponential bounds B1–B4) could be assigned to parallel agents, each given the exact signature to fill and the list of available lemmas.

Where agents excelled. "Fill this `sorry` given these lemmas"—tasks with a clear specification and known ingredients—compiled on the first attempt when delegated to agents. Most of the exponential bounds and the assembly step were completed this way.

Where agents struggled. "Find the right proof approach"—tasks requiring mathematical insight, such as the Strang commutator-scaling argument or the algebraic factorization for the step error—required human-driven exploration. The agent's tendency to attempt minor variations of a failing approach (rather than reconsidering the overall strategy) became a bottleneck for these tasks.

A cautionary pattern. As noted by Zhao et al. [11], LLM agents sometimes quietly strengthen hypotheses to make a proof trivial. The `sorry`-first workflow guards against this: since all signatures are fixed before proof filling begins, adding hypotheses breaks the type-checking of downstream theorems.

5.2 Mathlib ecosystem

Several gaps in Mathlib were encountered and filled during this project:

- $\|e^a\| \le e^{\|a\|}$ for general Banach algebras (`norm_exp_le`).
- $e^x - 1 - x \le x^2/2 \cdot e^x$ for $x \ge 0$ (`exp_sub_one_sub_bound_real`).

- $\|e^a - 1\| \le e^{\|a\|} - 1$ (`norm_exp_sub_one_le`).

These are natural candidates for upstreaming to Mathlib.

Working over a general field $\mathbb{K}$ (via `RCLike`) introduced friction with the calculus infrastructure, which assumes $\mathbb{R}$-module structure. Our pragmatic solution—working over $\mathbb{R}$ directly in the commutator-scaling files—is effective but means users must apply `restrictScalars` for $\mathbb{C}$ applications. A unified treatment would require additional typeclass instances in Mathlib.

5.3 Formalization-driven proof design

The formalization process influenced proof design in several ways:

Algebraic factorization. The standard approach to the step error—expanding $e^a\, e^b$ to second order and bounding cross-terms—requires tracking non-commutative multinomial coefficients. In Lean, this involves dozens of `norm_mul_le` and `gcongr` steps for each cross-term. The algebraic factorization (Section 3.4) splits the problem into two independent bounds, each handled by series comparison, reducing the proof length by approximately half. This factorization is arguably cleaner than the standard approach even on paper.

Duhamel without Gronwall. The commutator-scaling proof was designed with Lean feasibility in mind. The standard approach uses Gronwall's inequality or ODE uniqueness, which would require formalizing existence and uniqueness for linear ODEs in Banach spaces—theory not yet available in Mathlib. The Duhamel/FTC approach uses only the product rule and FTC-2, both already available.

`noncomm_ring` vs. `ring`. The `ring` tactic in Lean silently fails on non-commutative goals: it produces an unsolved subgoal rather than an error message. We discovered this only after extended debugging. The `noncomm_ring` tactic handles free non-commutative ring identities correctly but cannot use commutativity hypotheses (e.g., $[a, e^a] = 0$). Our workaround: use `set` to make exponentials opaque, rewrite commutativity as equalities, then apply `noncomm_ring` on the resulting free-ring identity.

5.4 Relation to quantum simulation

Our results are stated for general complete normed algebras, which include the bounded operators $B(\mathcal{H})$ on a Hilbert space as a special case. Specialization to finite-dimensional matrix algebras $M_d(\mathbb{C})$ is straightforward via Mathlib's `Matrix.instNormedRing`.

The commutator-scaling bounds are physically motivated by Hamiltonian simulation: for a lattice Hamiltonian $H = \sum_j h_j$ where each h_j acts locally, the commutator $[h_j, h_k]$ vanishes for distant terms. Our formalized bounds could serve as certified building blocks for verified resource estimation in quantum algorithms, once combined with gate-count analysis for individual exponentials.

6 Conclusion and Future Work

We have presented the first machine-checked formalization of the Lie–Trotter product formula, together with its higher-order extensions and commutator-scaling error bounds, in Lean 4 with the Mathlib library. The development comprises over 4 300 lines with zero `sorry` axioms and covers:

- First-order Lie–Trotter convergence at rate $O(1/n)$;
- Symmetric Strang splitting at rate $O(1/n^2)$;

- Multi-operator generalizations of both formulas;
- A fourth-order Suzuki integrator;
- First- and second-order commutator-scaling bounds via a novel Duhamel/FTC technique.

The Duhamel approach to commutator scaling—avoiding ODE uniqueness in favor of the fundamental theorem of calculus—may be of independent interest as a proof strategy for related product formula bounds.

Future directions. Several extensions are natural:

1. *Mathlib contributions.* The exponential norm bounds (`norm_exp_le`, `norm_exp_sub_one_le`, `exp_sub_one_sub_bound_real`) are general-purpose lemmas suitable for upstreaming to Mathlib.
2. *Tight higher-order Suzuki bounds.* Our fourth-order Suzuki integrator currently has an $O(1/n^2)$ bound; the tight $O(1/n^4)$ bound requires formalizing the parity-based cancellation argument of Suzuki [7].
3. *Baker–Campbell–Hausdorff formula.* The BCH formula $\log(e^A e^B) = A + B + \frac{1}{2}[A, B] + \cdots$ is closely related to the Trotter error expansion. A partial scaffold exists in our repository.
4. *Specialization to quantum systems.* Connecting our abstract bounds to concrete gate-count estimates for Hamiltonian simulation on quantum computers would bridge the gap between formal mathematics and verified quantum algorithms.
5. *Unbounded operators.* The original Trotter–Kato theorem applies to strongly continuous semigroups on Banach spaces, covering unbounded generators. Formalizing this extension would require substantial additional functional analysis infrastructure in Mathlib.

More broadly, our project demonstrates that substantial results in operator analysis—involving non-commutative algebra, power series manipulation, Fréchet calculus, and Bochner integration—are within reach of current proof assistant technology. We hope this work encourages further formalization of the mathematical foundations underlying quantum algorithms.

Acknowledgments

The Lean formalization was developed with assistance from Claude Code (Anthropic), which was used both as a coding assistant for filling proof obligations and as a writing assistant in the preparation of this manuscript.

References

[1] Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. Theory of Trotter Error with Commutator Scaling. *Physical Review X*, 11(1):011020, February 2021. ISSN 2160-3308. doi: 10.1103/PhysRevX.11.011020. URL `http://arxiv.org/abs/1912.08854`.

[2] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pages 625–635. Springer, 2021.

[3] Seth Lloyd. Universal Quantum Simulators. *Science*, 273(5278):1073–1078, August 1996. ISSN 0036-8075, 1095-9203. doi: 10.1126/science.273.5278.1073. URL `https://science.sciencemag.org/content/273/5278/1073`.

[4] Alex Meiburg, Leonardo A. Lessa, and Rodolfo R. Soldati. A formalization of the generalized quantum Stein's lemma in Lean, 2025.

[5] Yuanjie Ren, Jinzheng Li, and Yidi Qi. MerLean: An agentic framework for autoformalization in quantum computation, 2026.

[6] Gilbert Strang. On the construction and comparison of difference schemes. *SIAM Journal on Numerical Analysis*, 5(3):506–517, 1968.

[7] Masuo Suzuki. General theory of fractal path integrals with applications to many-body theories and statistical physics. *Journal of Mathematical Physics*, 32(2):400–407, February 1991. ISSN 0022-2488. doi: 10.1063/1.529425. URL `https://doi.org/10.1063/1.529425`.

[8] The Mathlib Community. The Lean mathematical library. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381, 2020.

[9] Joseph Tooby-Smith. Formalizing the stability of the two Higgs doublet model potential into Lean: identifying an error in the literature, 2026.

[10] Hale F. Trotter. On the product of semi-groups of operators. *Proceedings of the American Mathematical Society*, 10(4):545–551, 1959.

[11] Tianrun Zhao and Nengkun Yu. Formalizing CHSH rigidity in Lean 4. *arXiv preprint arXiv:2604.03884*, 2025.